

Lab 10: Antireversing Techniques

In this lab, you will modify a license key checker and protect it from reverse engineering.

Purpose

To learn about and apply antireversing techniques to existing source code in order to protect a sensitive algorithm.

Procedure

You will need access to the following materials:

- A computer with a working, licensed copy of Windows or Linux.
- A suite of reverse-engineering tools likely consisting of at least one static and one dynamic analysis tool.
- A software development environment for the C/C++ programming language.
- A copy of this lab's license key checker.

You have been provided with the source code for a simple license key checker. You can think of this as a level 0 crack me. It contains an algorithm used to check a license key given a particular user's name. Your task is to create a variant of this algorithm, and to protect it from analysis by applying two or more antireversing techniques.

1. The provided source code should compile in Visual Studio on Windows or GCC on Linux. Compile the code in its Debug and Release configuration. You may notice, the Debug configuration includes the key generator.
2. Analyze the program. Briefly describe the key checking algorithm in your report.
3. Program your own key checking program. It may be a variant on the one provided, or you may create your own from scratch. You may program it for Windows or Linux, but it should follow the same interface as the provided one. The first parameter should be the user name, and the second should be a key. If the key is correct, the program should clearly indicate such, if it is not, the program should clearly indicate such.
4. Apply two or more schemes to protect your algorithm from being discovered. Remember, it's impossible to completely protect your code, but you can make things difficult. You may use techniques in the book (provided they still work on an OS newer than Windows 7), you may research techniques online, or you may develop your own.
5. Demonstrate and document the effect of each technique you apply, i.e., show that it creates some additional hurdle for a reverse engineer. If it helps with division of labor, you may, instead of submitting one variant with two techniques, submit two variants each with one technique.

Tips and Caveats

- I hope to use your submissions as specimen for Lab 12, so use discretion in the difficulty of your techniques. I will also provide my own for Lab 12 with a moderately advanced technique.
- You will submit source code and a compiled binary. For Lab 12, all binaries will be posted without source.
- You may find it helpful to use inline assembly.
- Depending on how advanced your technique is, you may need to enable write on the .text section. Google how to modify section attributes using your toolchain's linker and/or OS API.

- Please do not use pre-made protections, e.g., UPX packer, MOV-only compiler.

Report

Completion of this lab must be substantiated by a report prepared by the group. It is up to each group to divide its responsibilities fairly. The report should provide sufficient detail that another person could repeat your exact or similar experiments.

Please include an archive of any source code and resulting binaries, if applicable, in your electronic submission.

Grading

The report should describe the purpose of the lab, the procedure, any assumptions, problems, or hypotheses, and the final results. Mostly, I'll be looking for a proper problem statement, a structured procedure, and for a reasonably correct solution demonstrated by your results.

This lab is due before class next week.

Rubric

points	Accomplishment
3	Report is well-written (organized, detailed, illustrative)
1	Report describes the modifications to the license checking algorithm, including a correct example username/key combination
2	Report describes two protection mechanisms applied to your variant
2	Source correctly implements the mechanisms described
1	Source compiles without error on the specified platform
1	Resulting binary runs without error